

Relationship Between Player Salary and Player Performance: Which NBA Teams Get the Most Bang for Every Buck

DSCI-510 Final Report | Alexander Sanchez | University of Southern California | [GitHub](#)

1. Motive/Rationale

Unlike the MLB, which can spend as much as it wants on player acquisitions, the NBA operates under a salary cap, which heavily influences how teams allocate their money when building a team. While teams want star players, they can't just spend 100 million dollars for two players, and spend the remaining 50 million for the remaining roster; this doesn't even consider the possibility of your star player getting injured, removing him from the entire season, like the case with Tyrese Haliburton. The teams that win championships are often those that find undervalued contributors: players who outperform their contracts. This project investigates which NBA teams have the highest salary efficiency and which players are most undervalued compared to their contracts.

2. Description of Data Source

Basketball-Reference (Player Performance Data and Years of Experience)

Two datasets were collected from Basketball-Reference for the 2024–25 NBA season: per-game statistics (points, rebounds, assists, shooting percentages, etc.) and advanced metrics (Player Efficiency Rating, Win Shares, WS/48, Box Plus-Minus, and Value Over Replacement Player). Both were scraped using `pandas.read_html`, targeting specific table IDs. A 4-second delay was added between requests to avoid being rate-limited. Each raw dataset contains 737 rows, reduced to 571 after cleaning.

https://www.basketball-reference.com/leagues/NBA_2025_advanced.html

https://www.basketball-reference.com/leagues/NBA_2025_per_game.html

Basketball-Reference (Player's Years of Experience)

Years of experience for each player were scraped from Basketball-Reference's per-team roster pages for the 2024–25 season. We iterated over all 30 team URLs, extracting each team's roster table using `pandas.read_html` and tagging each row with its team abbreviation. We then concatenated the results into a single dataset. A 1.2-second delay was added between requests to avoid rate limiting across the 30 pages.

<https://www.basketball-reference.com/teams/>

ESPN (Player Salary Data)

The original proposal planned to use HoopsHype salary data via their internal GraphQL API; however, the API consistently returned 401 Unauthorized errors. As a result, ESPN's publicly accessible salary pages were used instead, providing the same information without authentication. Player salary data for the 2024–25 season (player name, position, team, and salary) was scraped from 14 pages using requests and `pandas.read_html`, with a 1.2-second delay between pages to avoid being blocked. The raw dataset contains 543 rows, which were reduced to 492 after processing.

https://www.espn.com/nba/salaries/_/year/2025

UPDATE: The original requests-based scraper was fully functional as of May 3 (the old deadline). While testing reproduction, ESPN implemented AWS WAF bot protection, which blocks standard HTTP requests. The change likely occurred between May 4 and 9. The scraper was updated to use Playwright, which launches a headless Chromium browser to handle this automatically. The raw-scraped data remained unchanged, so the cleaning script remained unchanged.

NBA API (Team Performance Data)

Team-level standings data (wins, losses, win percentage, and conference rankings for all 30 NBA teams) were retrieved using the `nba_api` Python package, which accesses NBA.com's internal endpoints without authentication. Specifically, the `LeagueStandingsV3` endpoint was called for the 2024–25 regular season. This source required no structural cleaning, so both the raw and processed files contain 30 teams (30 rows excluding the column header).

https://github.com/swar/nba_api

Data Cleaning and Processing: Several preprocessing steps were necessary to ensure consistency across datasets:

- **Name standardization:** Player names were normalized, such as removing accents: “Jokić” → “Jokic”. Data from Basket-Ball Reference came with accented names, while ESPN salaries didn’t. Decided to remove the accents for easier merging.
- **Handling traded players:** Rows representing multi-team totals (e.g., “TOT”, “2TM”) were filtered out from our analysis
- **Salary cleaning:** Salary strings (e.g., “\$55,761,216”) were converted into numeric values
- **Team name alignment:** Team names were mapped to consistent abbreviations across all datasets. Basketball-Reference teams were abbreviated, while ESPN salaries listed only team names combined with their respective home cities. The team standings from nba_api had team names and cities as separate columns; I merged them, similar to ESPN's salaries team names. Used the same mapping logic for both ESPN salaries and team standings for team abbreviations.
- **Duplicate removal:** Duplicated rows were removed to ensure one record per player-team combination

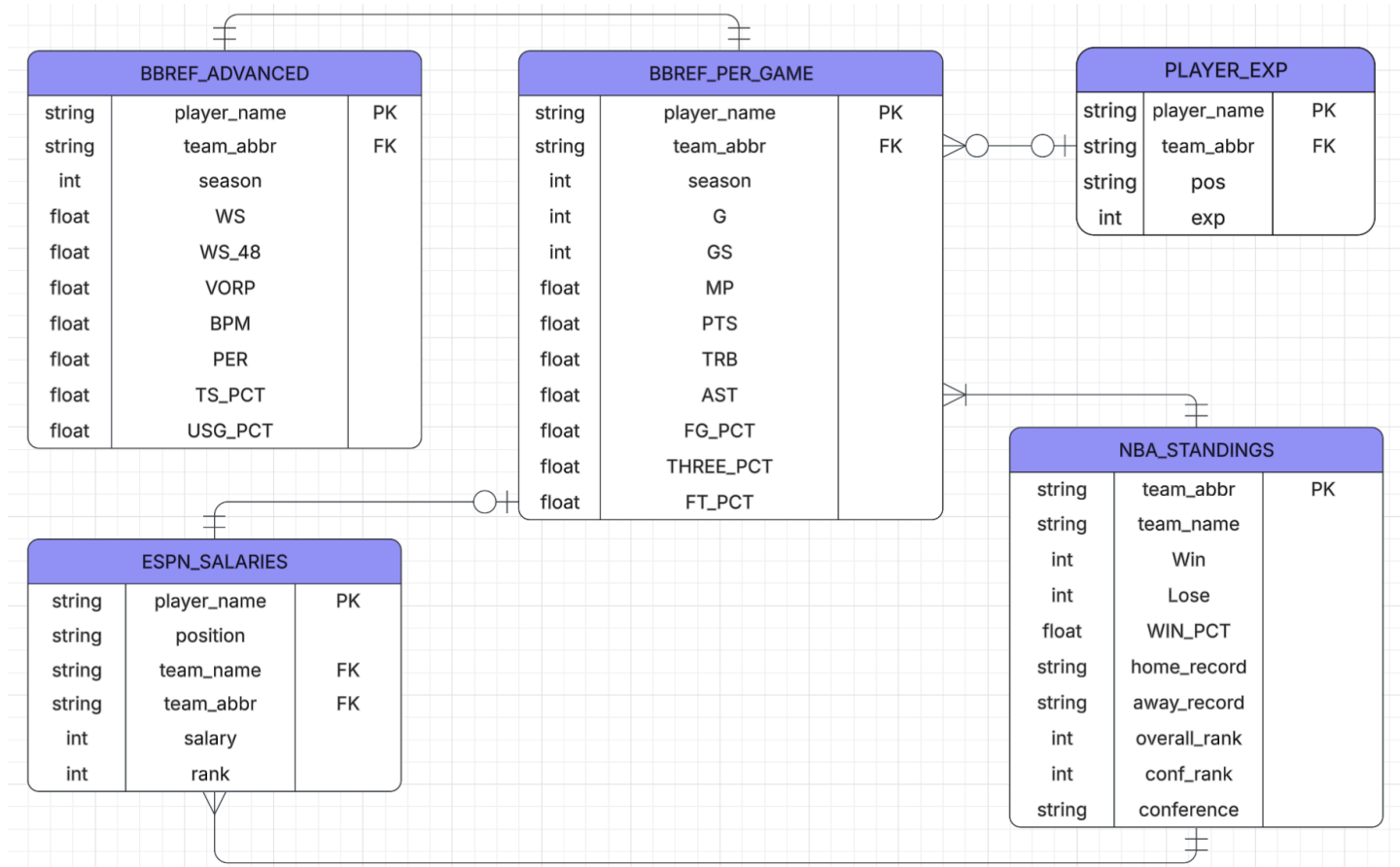
Changes from Original Proposal: The original project proposal specified using HoopsHype salary data via a GraphQL API. However, during implementation, the API consistently returned 401 Unauthorized errors, preventing access to data.

Resolution: The salary data source was replaced with ESPN's publicly accessible salary pages. ESPN provides the same salary information without requiring authentication. This change did not affect the project's overall structure or goals, as the same type of salary data was obtained.

Challenges and Resolutions

1. **API Access Issues (HoopsHype)**
 - a. Challenge: GraphQL API returned authentication errors (401 Unauthorized)
 - b. Resolution: Switched to ESPN as an alternative data source
2. **Web Scraping Constraints (ESPN)**
 - a. Challenge: ESPN rate-limits pagination
 - b. Resolution: Implemented pagination and added delays between requests to avoid blocking
3. **Inconsistent Data Formats**
 - a. Challenge: Differences in player names and team naming conventions across sources
 - b. Resolution: Standardized names and mapped teams to consistent abbreviations
4. **Traded Players and Duplicate Rows**
 - a. Challenge: Players appearing multiple times (e.g., “TOT”, “2TM”). Indicated the player played with two or more teams during the season
 - b. Resolution: Removed players who played with multiple players for the season
5. **ESPN AWS WAF Bot Protection**
 - a. Challenge: ESPN implemented AWS WAF bot protection, which blocks standard HTTP requests and requires JavaScript to complete a browser challenge
 - b. Resolution: The scraper was updated to use Playwright, which launches a headless Chromium browser to handle this automatically

3. Integrated Data Model



The final data model integrates multiple datasets at both the player and team levels to support analysis of salary efficiency in the NBA. The model combines player performance statistics from Basketball-Reference, salary data from ESPN, and team-level performance metrics from the NBA API. These datasets are linked using player names and team abbreviations.

At the player level, the BBREF_PER_GAME and BBREF_ADVANCED tables contain different types of performance statistics. These tables are joined using (player_name, team_abbr) and have a one-to-one relationship.

The ESPN_SALARIES table provides salary and position information. It is also joined using (player_name, team_abbr). Most players have one matching salary record, though some may be missing, resulting in a zero-or-one relationship.

At the team level, the NBA_STANDINGS table includes team performance metrics such as wins, losses, and rankings. This table is linked to player data via team_abbr, creating a one-to-many relationship because each team can have multiple players.

The PLAYER_EXP table contains each player's years of experience and is joined to the master dataset on (player_name, team_abbr) using a left join, preserving all players even if no experience record is found. This creates a zero-or-one relationship, since a small number of players may be missing experience data.

All tables are merged into a single dataset for analysis. During integration, data is cleaned by standardizing player names, aligning team abbreviations, and removing duplicate records.

4. Analysis/Visualization

Metrics

- **WS (Win Shares):** an estimate of the number of wins a player contributed to their team
- **WS/48:** Win Shares normalized to 48 minutes (a full game), allowing fair comparison across players with different playing times
- **BPM (Box Plus-Minus):** a per-100-possession estimate of a player's contribution relative to a league-average player
- **VORP (Value Over Replacement Player):** total value added relative to what a replacement-level player would provide over the same minutes

- **PER (Player Efficiency Rating):** a per-minute measure of overall player productivity, normalized so the league average is 15
- **USG% (Usage Rate):** the percentage of team plays a player uses while on the floor
- **TS% (True Shooting %):** a shooting efficiency metric that accounts for 2-pointers, 3-pointers, and free throws
- **Win_PCT:** Team wins divided by total games played
- **exp:** Years of experience

My Derived Metrics

- **WS/\$1M:** Win Shares earned per million dollars of salary, your core efficiency measure
- **VORP/\$1M:** same idea applied to VORP
- **Expected salary:** what the regression model predicts a player should earn given their stats, used as a baseline to identify over- or underpaid players
- **Salary difference:** the gap between expected and actual salary; negative means the player is underpaid relative to their production

Limitations on the derived metrics: WS/\$1M and VORP/\$1M are computed solely for this analysis and inherit the limitations of their underlying metrics. Both treat salary as a simple divisor, ignoring contract context (rookie deals, max contracts, age, injury risk, and years remaining), which influence NBA compensation independent of on-court performance.

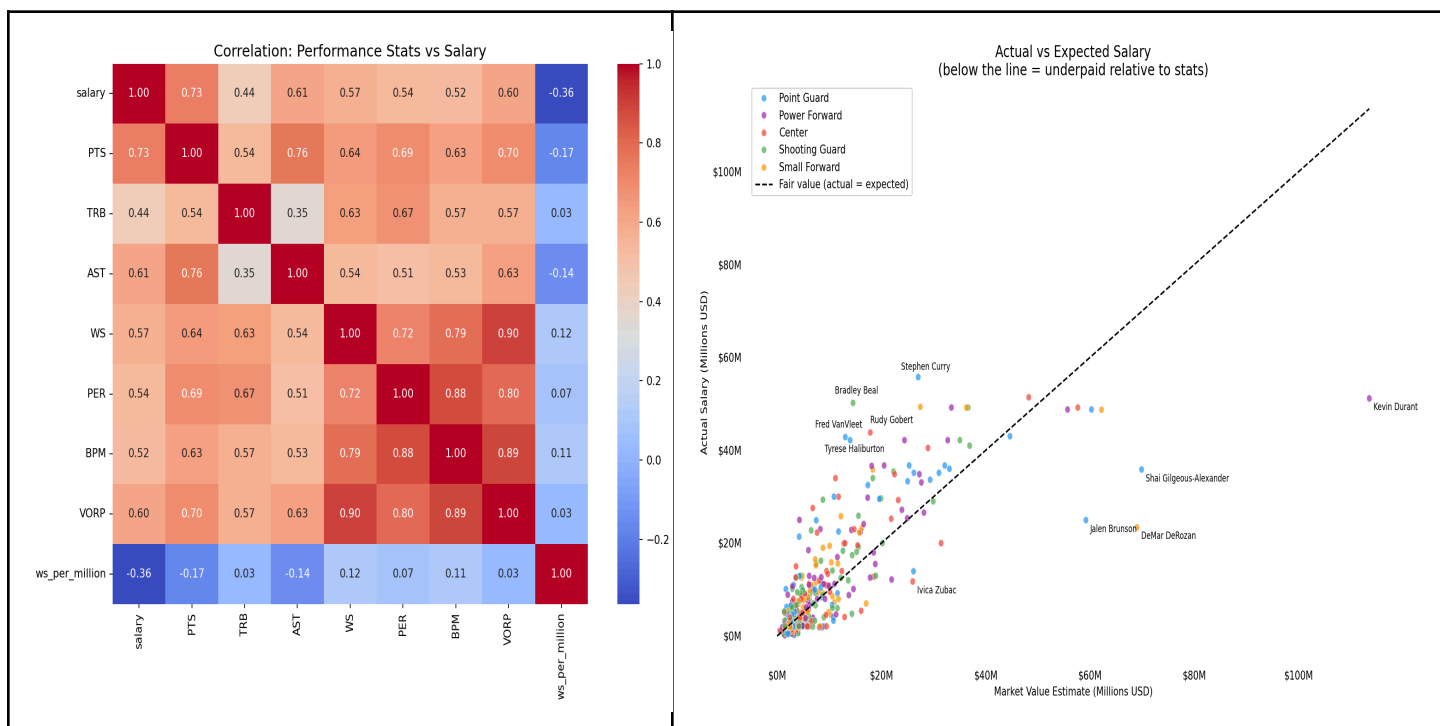
Analysis Technique

Derived metrics were computed to measure efficiency: Win Shares per million dollars (WS/\$1M) for both players and teams, Value Over Replacement Player per million (VORP/\$1M), and an expected salary estimate produced by a per-position log-linear regression model.

A separate regression model was fit for each position (Point Guard, Shooting Guard, etc.) using nine performance features: Win Shares, VORP, Box Plus-Minus (BPM), points, assists, rebounds, usage rate, age, and games played. Salary was log-transformed before fitting to prevent the model from producing negative predictions. The purpose of the regression was not to build a perfect predictive model or to interpret individual coefficients, but to establish a baseline for what a player should earn based on their on-court performance. Players earning below their expected salary are considered undervalued relative to their stats.

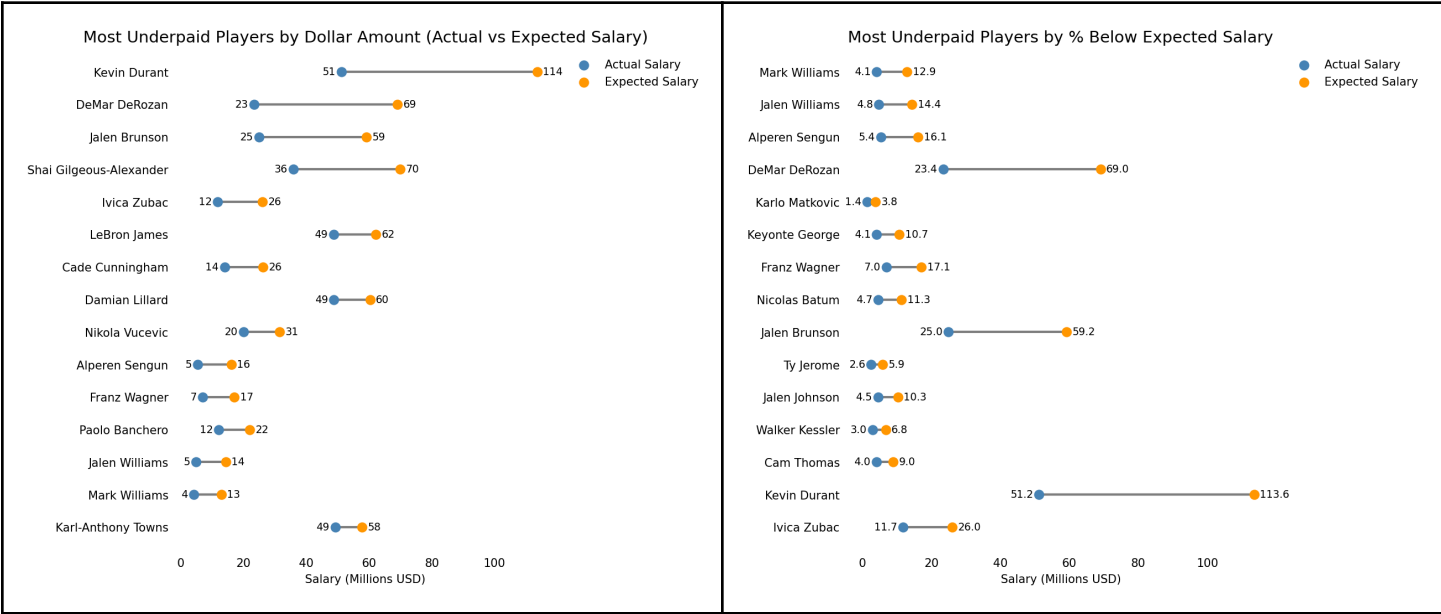
A correlation analysis was then conducted using a correlation matrix of salary and key performance metrics (points, rebounds, assists, WS, PER, BPM, and VORP) to identify which statistics most strongly correlate with compensation.

Finally, a gap analysis ranked players by the difference between expected and actual salary to identify the most undervalued players in the league, and a team-level efficiency analysis used WS/\$1M to evaluate how effectively franchises convert payroll into on-court performance.

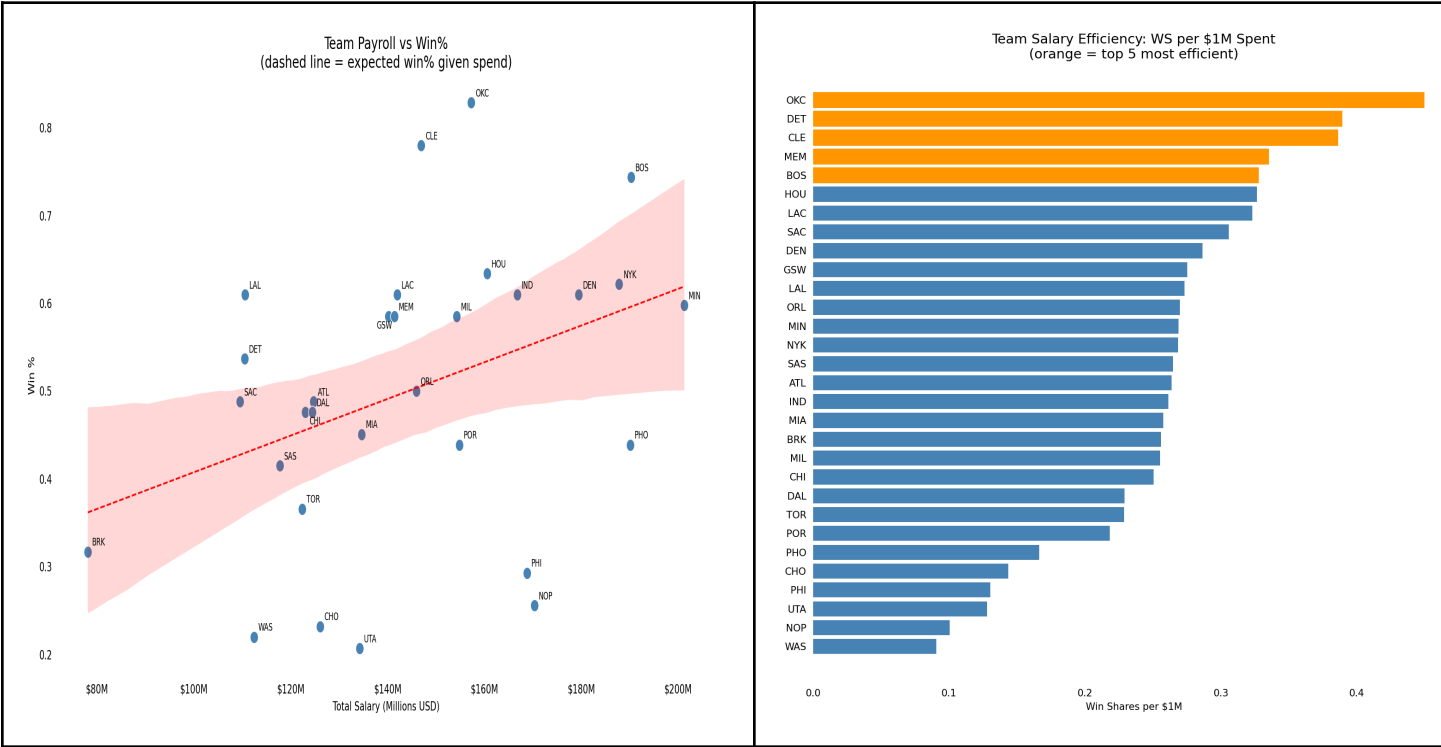


The correlation heatmap shows Pearson correlation coefficients among salary, traditional stats (points, rebounds, assists), advanced metrics (Win Shares, PER, BPM, VORP), and our derived WS/\$1M efficiency metric, on a red-to-blue scale. Points per game shows the strongest correlation with salary ($r = 0.73$), suggesting that scoring is the most rewarded skill in the NBA market. Advanced metrics such as WS, VORP, BPM, and PER are moderately correlated with salary (0.52–0.60), indicating that teams reward overall contribution, though not as strongly as raw scoring. Notably, WS/\$1M is negatively correlated with salary ($r = -0.36$), meaning higher-paid players tend to deliver fewer Win Shares per dollar.

This scatter plot maps each player's model-predicted salary against their actual salary, with points colored by position. The dashed diagonal represents fair value: players above it earn more than their stats justify, while those below it are underpaid relative to their performance. The five most overpaid and five most underpaid players earning at least \$10M are annotated with their names to anchor the visual. The most overpaid players: Stephen Curry, Bradley Beal, Rudy Gobert, Fred VanVleet, and Tyrese Haliburton. These players appear above the line, are underperforming to their contract. The most underpaid players: Kevin Durant, Shai Gilgeous-Alexander, Jalen Brunson, DeMar DeRozan, and Ivica Zubac. These players fall well below the line, indicating that their on-court performance outpaces their contract. The cluster of points near the origin follows the diagonal closely, suggesting that lower-paid players are generally priced more accurately by the market than higher-paid ones, a pattern consistent with the negative WS/\$1M correlation observed in the heatmap.



Both charts are dumbbell plots ranking the top 15 most underpaid players, using two different measures of undervaluation. The first ranks players by raw dollar difference, where Kevin Durant's gap of roughly \$63M dwarfs the rest of the list. The second ranks by percentage below the expected salary, with players on CBA-mandated minimum contracts filtered out; specifically, any player earning less than 110% of their experience-based minimum. This is intended to reflect a more accurate measure of market undervaluation rather than fixed rookie or veteran minimum deals. This is why younger players like Mark Williams, Jalen Williams, and Alperen Sengun lead the percentage chart despite their relatively low salaries: all three are first-round picks still on their rookie-scale contracts, which are fixed by draft slot, clear the CBA floor, but fall short of what their performance would command on the open market.



This scatter plot shows each of the 30 NBA teams as a labeled point. A regression line with a confidence interval is overlaid to illustrate the overall trend. Although there is a modest positive relationship between spending and winning, the confidence interval is wide, and the scatter is substantial, indicating that

payroll alone is a weak predictor of team success. Oklahoma City Thunder (OKC) and Cleveland Cavaliers (CLE) stand out as clear outliers above the trend line: both achieved high win percentages with relatively modest payrolls. Phoenix Suns (PHO) and New Orleans Pelicans (NOP) represent the opposite: high spending with poor results.

The horizontal bar chart ranks all 30 teams by WS/\$1M, with the top five most efficient franchises highlighted in orange. Oklahoma City Thunder (OKC) leads by a considerable margin, followed by Detroit Pistons (DET), Cleveland Cavaliers (CLE), Memphis Grizzlies (MEM), and Boston Celtics (BOS). The chart reveals an interesting split: Oklahoma City Thunder and Cleveland Cavaliers rank among both the most efficient teams and the teams with the highest win percentages, suggesting their rosters are built on well-valued contracts. Detroit's presence in the top five is interesting; they ranked highly efficient despite a middling record.

5. Conclusion

The NBA salary cap creates a zero-sum resource allocation problem, and the data suggest that the teams that solve it best are not necessarily the ones spending the most. Scoring remains the market's most rewarded skill ($r = 0.73$ with salary), while advanced metrics correlate more modestly. WS/\$1M is negatively correlated with salary ($r = -0.36$), meaning higher-paid players tend to deliver less value per dollar. At the team level, payroll alone is a weak predictor of winning: OKC and Cleveland demonstrate that elite records are achievable on modest payrolls, while Phoenix and New Orleans show the opposite.

The player-level analysis reinforces the same conclusion from a different angle: the league's most undervalued players are largely young first-round picks on rookie-scale contracts, whose fixed compensation bears little relationship to their actual performance. In a cap-constrained league, the clearest path to salary efficiency is simply drafting the right player, but that raises the most challenging question: which college basketball player will succeed in the NBA?

6. Further Work

The analyses presented here offer a useful first pass at assessing salary efficiency, but several limitations point to meaningful directions for future work. The most significant limitation is that the regression model treats salary as a pure function of on-court statistics, ignoring the many non-performance factors that shape NBA contracts in practice. Injury history, contract-year effects, age curves, market size, and a player's perceived franchise value all influence what teams pay, and none of these are captured here. A player like Bradley Beal, who appears overpaid relative to his recent stats, signed his max extension at the peak of his on-court production; the contract reflects a bet on future performance, not a valuation of the present. Longitudinal performance data reveal how a player evolves over time, but only if the game they're evolving in stays the same. The NBA has already shown how quickly analytics can reshape the game: the mid-range jumper, once a staple of offensive strategy, has nearly vanished as teams have optimized their strategies around shot value. If player valuation follows a similar trajectory, longitudinal data risks encoding outdated market logic rather than improving the model. More data is not always better; as the league continues to evolve, the most recent seasons may carry more signal than a decade of historical averages.